

Contents

Chapter 8: Architecture: Designing AI Systems That Scale	1
1. Architecture Is the Management of Change	2
2. Modularity	3
3. Cohesion and Coupling	4
4. Interfaces Are Architectural Contracts	6
5. Dependency Inversion	7
6. Service Boundaries	8
7. APIs Define Architectural Boundaries	9
8. State Management	10
9. Event-Driven Architecture	12
10. Redesigning the Week 1 Application	13
11. What Happens at 1,000 Users?	15
12. What Happens at 1 Million?	17
13. Scale Is More Than Throughput	18
14. Avoid Premature Microservices	19
15. Architecture as a Set of Tradeoffs	20
16. The Architecture Review	21
17. Exercise: Architecture Under Pressure	22
Architecture A — Prototype	22
Architecture B — 1,000 users	22
Architecture C — 1 million users	23
18. The Architectural Mindset	23
19. Key Takeaways	24

Chapter 8: Architecture: Designing AI Systems That Scale

In the first week, you learned how to build an AI application.

Now the question changes.

Building an application that works is relatively easy. Designing an application that **continues to work as its load, complexity, data volume, failure rate, and organizational importance increase** is architecture.

For traditional software, architecture has always been about managing complexity and change. AI systems make the problem more difficult because they contain probabilistic components, expensive inference, external model providers, asynchronous operations, rapidly changing context, and state that may exist simultaneously in databases, conversation histories, vector stores, caches, and agent execution graphs.

The architecture of an AI system therefore determines much more than where the code lives.

It determines:

- where state lives
- where decisions are made
- where failures occur
- where latency accumulates
- where costs are incurred
- where security boundaries exist
- where components can evolve independently
- where AI behavior can be evaluated
- and ultimately, whether the system can scale without becoming unmanageable

The central exercise for today is simple:

Take the Week 1 application and redesign it as a system that could plausibly become production infrastructure.

Then ask two increasingly uncomfortable questions:

What happens if this application gets 1,000 users?

And then:

What happens at 1 million?

The point is not to predict the exact architecture required at either scale.

The point is to learn how to **reason about architectural consequences before they become production failures.**

1. Architecture Is the Management of Change

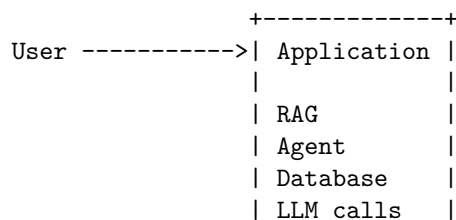
A useful definition of software architecture is:

Architecture is the set of structural decisions that determine how a system can change.

This is more useful than thinking of architecture as boxes and arrows.

Consider two implementations of the same research assistant.

System A



```
      | Documents |  
      | Evaluation |  
+-----+
```

Everything is inside one application.

It may work perfectly.

But now suppose you want to:

- replace the vector database
- add asynchronous document ingestion
- introduce caching
- support multiple LLM providers
- add enterprise authentication
- run evaluations automatically
- isolate expensive workloads
- introduce rate limits
- scale document processing independently
- add a second type of agent

The architecture now determines how difficult each change will be.

A better architectural question is therefore not:

“Is this architecture elegant?”

It is:

“What kinds of change does this architecture make cheap or expensive?”

This leads directly to the first architectural principle:

Optimize architecture for the changes the system is likely to experience.

2. Modularity

Modularity means decomposing a system into components with relatively well-defined responsibilities.

The objective is not simply to create more modules.

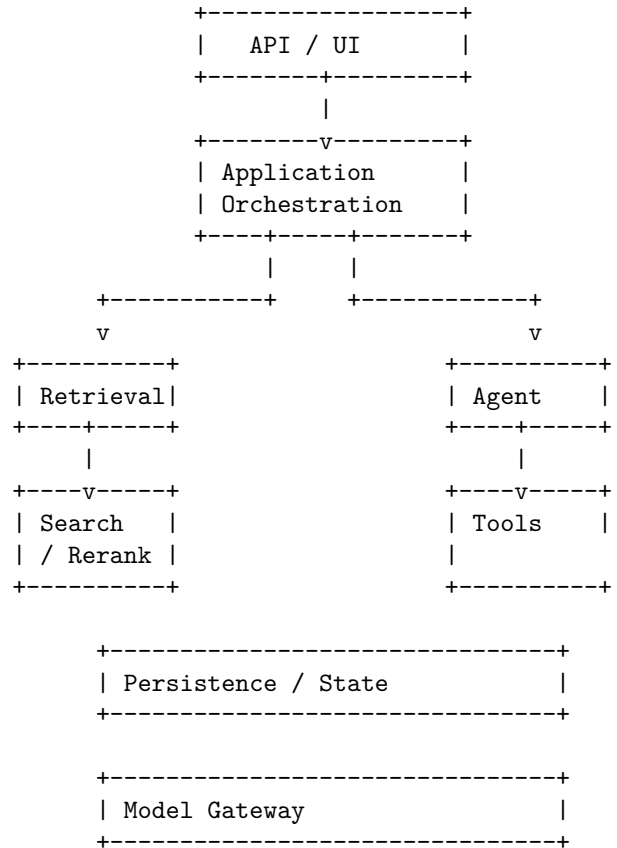
Excessive decomposition can be just as harmful as insufficient decomposition.

A useful module should provide:

1. **a coherent responsibility**
2. **a stable interface**
3. **controlled dependencies**
4. **independent reasoning**

5. a meaningful boundary for testing

For the Week 1 research assistant, a reasonable decomposition might be:



Notice something important.

The architecture does **not** require every box to become a microservice.

These may initially be Python modules in a single process.

That is perfectly reasonable.

Modularity and distribution are different concepts.

A modular monolith can be substantially better engineered than a collection of poorly designed microservices.

3. Cohesion and Coupling

Two of the most important architectural concepts are **cohesion** and **coupling**.

Cohesion Cohesion measures how strongly the responsibilities within a component belong together.

High cohesion is desirable.

For example:

RetrievalService

```
retrieve()
rerank()
construct_context()
```

These operations are conceptually related.

Compare that with:

UtilityService

```
retrieve()
authenticate()
charge_user()
generate_embeddings()
send_email()
calculate_latency()
```

This is a classic “miscellaneous service.”

Its apparent simplicity hides architectural disorder.

Coupling Coupling measures how strongly components depend upon each other.

Low coupling is generally desirable.

For example:

Application

|
v

Retriever interface

```
|
+-- VectorRetriever
+-- HybridRetriever
+-- MockRetriever
```

The application depends on the **abstraction**, not the implementation.

This allows the retrieval mechanism to change without rewriting the application.

The architectural goal can therefore be summarized as:

High cohesion within components; low coupling between components.

4. Interfaces Are Architectural Contracts

An interface defines what one component promises to another.

For example:

```
class Retriever(Protocol):

    def retrieve(
        self,
        query: str,
        *,
        top_k: int
    ) -> list[Document]:
        ...
```

The application does not need to know whether retrieval uses:

- PostgreSQL
- Elasticsearch
- a vector database
- BM25
- embeddings
- a remote retrieval API
- or an experimental neural retriever

It knows only the contract.

This has an important consequence:

Interfaces turn implementation choices into replaceable decisions.

Interfaces are particularly valuable in AI systems because AI infrastructure changes rapidly.

You may replace:

```
OpenAI
↓
Anthropic
↓
local model
↓
specialized model
```

↓
`multi-model router`

If model invocation is scattered throughout the application, this becomes an architectural migration.

If the system exposes:

```
class ModelProvider:
    def generate(request) -> ModelResponse:
        ...
```

the change becomes substantially more localized.

5. Dependency Inversion

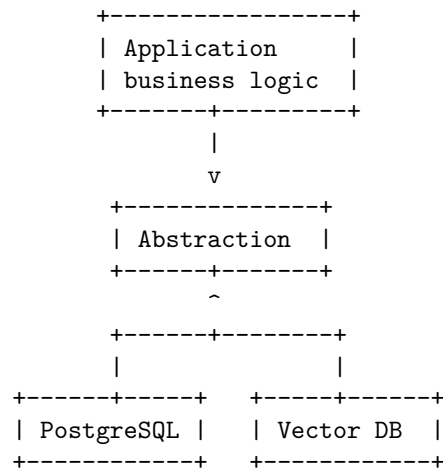
Dependency inversion is the natural extension of interface-driven architecture.

The traditional dependency pattern looks like:

```
Application
|
v
Concrete implementation
|
v
Infrastructure
```

The application becomes tightly coupled to infrastructure.

Dependency inversion instead produces:



The high-level policy depends on abstractions.

Infrastructure implements those abstractions.

This is particularly useful for testing.

The production system might use:

```
RealModelProvider
RealRetriever
RealDatabase
```

while the evaluation environment uses:

```
DeterministicModelProvider
FixtureRetriever
TestDatabase
```

The core application logic remains unchanged.

6. Service Boundaries

At some point, modularity becomes a question of **deployment boundaries**.

Should the system remain one process?

Should retrieval become a service?

Should document ingestion become asynchronous?

Should model inference be separated?

There is no universal answer.

A useful heuristic is:

Create a service boundary when independent scaling, deployment, ownership, security, reliability, or resource isolation provides meaningful value.

For example, document ingestion is often a strong candidate for asynchronous processing.

Instead of:

```
Upload document
  |
  v
Parse
  |
  v
Chunk
  |
  v
```



```

Embed
  |
  v
Index
  |
  v
Return response

use:

Upload
  |
  v
Object Storage
  |
  v
Queue
  |
  v
Ingestion Workers
  |
  +-- Parse
  +-- Chunk
  +-- Embed
  +-- Index

```

The user does not need to wait for the entire pipeline.

More importantly, ingestion can now scale independently from interactive queries.

7. APIs Define Architectural Boundaries

An API is not merely a mechanism for HTTP communication.

It is a **contract between independently evolving components**.

For example:

POST /v1/query

might accept:

```

{
  "conversation_id": "abc",
  "query": "What does the paper conclude?",
  "options": {
    "include_sources": true
  }
}

```

```

    }
  }
  and return:
  {
    "answer": "...",
    "sources": [...],
    "confidence": 0.82,
    "trace_id": "..."
  }

```

The API should define:

- request schema
- response schema
- error semantics
- authentication
- authorization
- versioning
- idempotency
- rate limits
- timeout expectations

AI systems introduce additional concerns.

For example, an API may need to expose:

```

model
token usage
latency
retrieval metadata
tool execution status
uncertainty
citations
evaluation metadata

```

But exposing internal implementation details creates coupling.

Good APIs therefore distinguish between:

what the consumer needs to know

and

how the system happens to implement it.

8. State Management

State is one of the most easily misunderstood aspects of AI architecture.

A conversational AI system may contain several different forms of state:

```
User state
|
+-- identity
+-- preferences
+-- permissions

Conversation state
|
+-- messages
+-- summaries
+-- active task

Agent state
|
+-- plan
+-- tool results
+-- intermediate artifacts
+-- execution status

Knowledge state
|
+-- documents
+-- embeddings
+-- indexes

System state
|
+-- jobs
+-- metrics
+-- configuration
```

These states have different lifetimes and consistency requirements.

Treating all of them as “conversation history” creates architectural problems.

A robust system explicitly distinguishes:

```
Ephemeral state
    ↓
Process memory

Session state
    ↓
Redis / database

Durable state
```

↓
Database / object storage

Derived state
↓
Vector indexes / caches

Execution state
↓
Workflow store

This becomes critical when scaling horizontally.

If a request can arrive at any server:

```
          +-- Server A
User -----+-- Server B
          +-- Server C
```

the application cannot depend on:

```
conversation_state = {}
```

inside one process.

The state must either be externalized or the routing architecture must explicitly guarantee affinity.

In most production systems, durable state should be externalized.

9. Event-Driven Architecture

Not every operation should happen synchronously.

AI applications frequently contain long-running operations:

- document processing
- embedding generation
- batch evaluation
- model fine-tuning
- report generation
- agent workflows
- asynchronous tool execution

An event-driven architecture decouples producers from consumers.

```
+-----+
| API   |
+-----+
      |
```



```

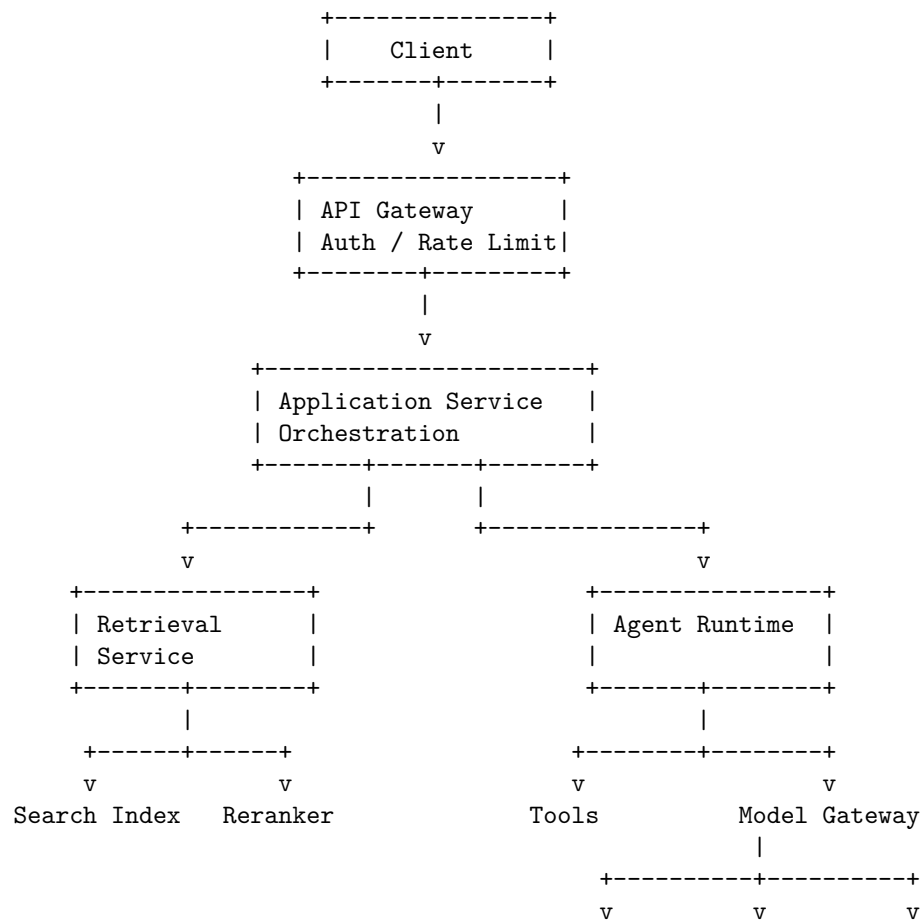
User
|
v
Web/API
|
v
Application
+-- RAG
+-- Agent
+-- Database
+-- LLM
+-- Tools

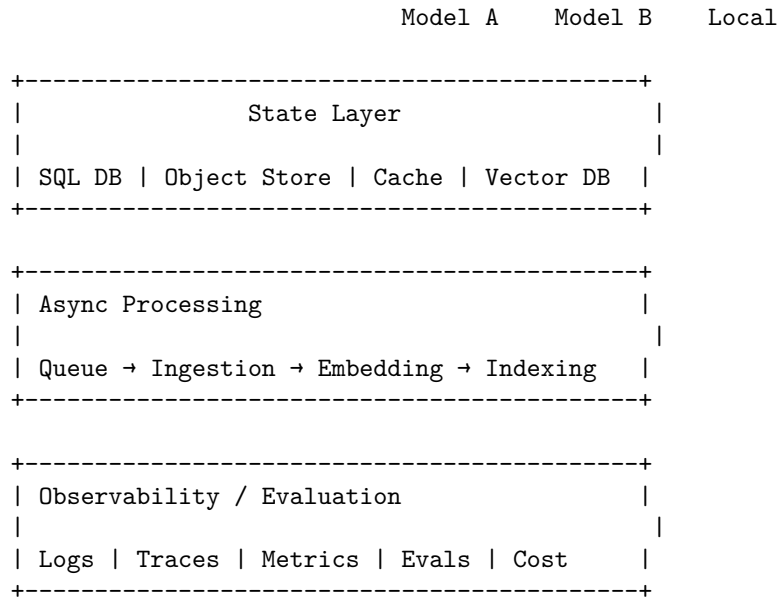
```

That is a reasonable prototype.

Now redesign it.

A more mature architecture might look like:





This is no longer merely an application.

It is an **AI system architecture**.

11. What Happens at 1,000 Users?

The first scaling exercise is deliberately modest.

Assume:

1,000 registered users
 100 concurrent users
 10 requests/sec peak

The important question is not whether the architecture can technically handle this.

The question is:

Where will it fail first?

Consider the request path:

```
Request
|
+-- authentication
+-- retrieval
+-- reranking
+-- model inference
```

```

+-- tool calls
+-- persistence
+-- response

```

Now identify bottlenecks.

Model inference LLM calls may dominate:

- latency
- cost
- throughput

Therefore introduce:

```

Model Gateway
|
+-- routing
+-- retries
+-- fallback
+-- caching
+-- quotas
+-- cost accounting

```

Retrieval Repeated queries may generate redundant work.

Introduce:

```

Query
|
v
Cache
|
+-- hit -----> result
|
+-- miss -----> retrieval

```

Database Connection pools become important.

API Rate limiting prevents one client from consuming the entire system.

Observability You need to know whether latency comes from:

API	20 ms
Retrieval	80 ms
Reranking	50 ms
LLM	900 ms
Tool	300 ms
DB	30 ms

Without distributed tracing, these numbers disappear into a single:

request = 1.4 seconds

Architecture therefore interacts directly with observability.

You cannot reliably operate what you cannot decompose.

12. What Happens at 1 Million?

Now increase the scale by three orders of magnitude.

Do not simply multiply the number of servers.

Reconsider the architecture.

At 1 million users, new questions appear.

Capacity What is the peak requests-per-second?

1,000,000 users

x

daily activity

x

requests/session

x

peak concentration

The important number is not registered users.

It is **concurrent workload**.

Cost Suppose every request invokes an expensive model.

Then:

1M users

x

requests/user

x

tokens/request

x

model price

can dominate the economics of the system.

Architecture must therefore become cost-aware.

Potential mechanisms include:

- model routing

- caching
- smaller models
- batching
- context reduction
- request deduplication
- asynchronous processing
- token budgets
- per-user quotas

Data At one million users, persistent state becomes substantial.

You may need:

Partitioning
Replication
Archival
Lifecycle policies
Read replicas
Index optimization
Object storage

Reliability A single dependency failure becomes unacceptable.

If the architecture depends entirely on one model provider:

```

Application
  |
  v
Model Provider
  X

```

the entire system fails when the provider fails.

A more resilient architecture may provide:

```

          Model Gateway
            |
      +-----+-----+
      v         v         v
Provider A Provider B Local

```

The gateway becomes a resilience boundary.

13. Scale Is More Than Throughput

A common architectural mistake is defining scalability as:

“Can it handle more requests?”

There are at least four dimensions of scale.

Computational scale Can the system process more work?

requests/sec
tokens/sec
documents/sec
jobs/sec

Data scale Can it manage more information?

users
documents
embeddings
conversations
artifacts
logs

Organizational scale Can more engineers modify the system without constantly interfering with each other?

This is where modularity becomes extremely important.

Complexity scale Can the system support more behaviors?

For example:

RAG
+
agents
+
tools
+
memory
+
multimodal input
+
automated evaluation
+
personalization

A system may scale computationally while failing organizationally or architecturally.

14. Avoid Premature Microservices

The natural reaction to architecture discussions is often:

“We should use microservices.”

That is not the lesson.

Microservices introduce:

- network latency
- distributed failure
- deployment complexity
- service discovery
- observability requirements
- data consistency problems
- operational overhead

A modular monolith can often handle surprisingly large workloads.

A good progression is:

Prototype
↓
Modular monolith
↓
Async workers
↓
Selective service extraction
↓
Distributed architecture

Extract a service when there is a reason.

For example:

“Document ingestion needs independent scaling and can tolerate asynchronous execution.”

That is an architectural argument.

“Microservices are modern” is not.

15. Architecture as a Set of Tradeoffs

There is no perfect architecture.

Every architectural decision trades one property against another.

For example:

Decision	Benefit	Cost
Cache	Lower latency/cost	Staleness/invalidation
Queue	Resilience/buffering	Asynchrony/complexity

Decision	Benefit	Cost
Replication	Availability/read scale	Consistency/storage cost
Microservice	Independent scaling	Distributed complexity
Local model	Cost/control	Operational burden
External model	Capability	Vendor dependency
Strong consistency	Correctness	Latency/throughput
Eventual consistency	Scalability	Stale reads
Abstraction	Replaceability	Additional indirection

Senior engineering is therefore not about memorizing architectural patterns.

It is about understanding **which tradeoff is appropriate for the system's constraints**.

16. The Architecture Review

For today's exercise, perform a formal architecture review of the Week 1 application.

Document at least:

1. Components What are the major modules?

API
Orchestration
Retrieval
Agent
Model Gateway
Persistence
Async Workers
Evaluation
Observability

2. Interfaces What contracts exist between them?

3. Dependencies Which components depend on which?

4. State Where does every important piece of state live?

5. Failure boundaries What happens if each dependency fails?

6. Scaling dimensions Which components scale independently?

7. Synchronous versus asynchronous work Which operations must happen during the request?

Which can become jobs?

8. Cost centers Where does money get spent?

9. Security boundaries Where are authentication, authorization, secrets, and sensitive data handled?

10. Evolution What happens when you replace:

- the model
- the vector database
- the retrieval algorithm
- the UI
- the agent framework
- the storage layer

The final question is the most revealing:

How many files must change to replace one major infrastructure component?

If replacing the model provider requires modifying 30 unrelated modules, the architecture has leaked infrastructure concerns into application logic.

17. Exercise: Architecture Under Pressure

Take your Week 1 application and create three architectures.

Architecture A — Prototype

Design the smallest system that works.

```
Client
  ↓
Application
  ↓
Database
  ↓
LLM
```

Architecture B — 1,000 users

Introduce only the boundaries that become necessary.

Consider:

- API gateway
- authentication
- rate limiting
- caching
- connection pooling
- asynchronous workers
- observability
- model gateway

Architecture C — 1 million users

Now reconsider everything.

Ask:

What must scale independently?

What must become asynchronous?

What must be cached?

What must be replicated?

What must be partitioned?

What can fail independently?

What must have a fallback?

What becomes too expensive?

What becomes a security boundary?

Then compare the three designs.

The goal is not to produce the most complicated architecture.

The goal is to understand **why the architecture changes as the system's constraints change.**

18. The Architectural Mindset

The progression from developer to systems engineer can be seen clearly here.

A developer asks:

“How do I implement this feature?”

A senior developer asks:

“Where should this feature live?”

An architect asks:

“What boundary should this feature cross?”

A systems engineer asks:

“What happens when this boundary fails, scales, changes, or becomes a bottleneck?”

AI engineering adds another dimension:

“What happens when the probabilistic component behaves differently from what we expected?”

That question connects architecture directly back to the topics from Week 1.

Architecture determines where you can:

- evaluate behavior
- observe failures
- constrain models
- isolate tools
- control context
- manage state
- enforce permissions
- control cost
- recover from failure

The architecture is therefore part of the AI system’s **reliability mechanism**.

19. Key Takeaways

1. **Architecture is fundamentally about managing change.** A good architecture makes important future changes cheap and local.
2. **Modularity is not the same as microservices.** Start with strong module boundaries; distribute components only when there is a concrete reason.
3. **High cohesion and low coupling are foundational architectural properties.**
4. **Interfaces are contracts.** They allow implementations to change without forcing the entire system to change.
5. **Dependency inversion separates application policy from infrastructure.** This is particularly valuable in AI systems where model providers, retrieval engines, and infrastructure evolve rapidly.

6. **State must be explicitly modeled.** Conversation state, agent execution state, knowledge state, and durable application state have different lifetimes and consistency requirements.
7. **Asynchronous architecture is essential for long-running AI workloads.** Queues provide buffering and enable independent scaling, but introduce distributed-systems concerns.
8. **APIs are architectural boundaries, not merely HTTP endpoints.** They define contracts between independently evolving parts of the system.
9. **Scaling is multidimensional.** Computational scale, data scale, organizational scale, and behavioral complexity all matter.
10. **At 1,000 users, bottlenecks become visible. At 1 million, architectural assumptions become economic and operational constraints.**
11. **Do not architect for scale you do not need—but do architect boundaries that preserve your ability to scale later.**
12. **The best architecture is not the most sophisticated architecture.** It is the simplest architecture that provides the required reliability, scalability, security, cost structure, and ability to evolve.

The central lesson of Chapter 8 is therefore:

Architecture is the engineering discipline of deciding where complexity should live—and designing the system so that complexity does not spread everywhere.

That becomes especially important in AI systems, because the model itself is only one component.

The real engineering challenge is building the system around it.